Applied Soft Computing Journal 154 (2024) 111306

Contents lists available at ScienceDirect

Applied Soft Computing

journal homepage: www.elsevier.com/locate/asoc

Conference

Meta-heuristic Algorithms for Double Roman Domination Problem

Himanshu Aggarwal, P. Venkata Subba Reddy *

Department of Computer Science and Engineering, National Institute of Technology Warangal, Hanamkonda, 506004, Telangana, India

ARTICLE INFO

Dataset link: https://math.nist.gov/MatrixMarket/matrices.html

Keywords:
Dominating set
Double roman domination
NP-hard
Genetic algorithm
Ant colony optimization algorithm

ABSTRACT

A variety of domination concepts have been defined to provide better routing and defense strategies under different constraints. A double Roman dominating function (DROMDF) on a simple, undirected graph G is a function $g : V \to \{0, 1, 2, 3\}$ such that every vertex $x \in V$ with $g(x) = 0$ is adjacent to at least two vertices y_1, y_2 with $g(y_1) = g(y_2) = 2$ or a vertex z_1 with $g(z_1) = 3$. Also, a vertex p with $g(p) = 1$ is adjacent to at least one vertex q_1 with $g(q_1) \geq 2$. $\gamma_{dR}(G)$, the double Roman domination number of G, is the smallest possible weight of all possible DROMDFs of G. Determining double Roman domination number of a graph is known to be NP-hard. Hence in this paper, we propose a genetic algorithm based approach for solving double Roman domination problem in which three heuristic algorithms have been proposed and problem specific crossover operator and a feasibility function has been developed. Further, we propose an ant colony optimization algorithm to solve double Roman domination problem. This paper provides an in-depth illustration of two algorithms for solving double Roman domination problem. Effectiveness of the proposed meta-heuristic algorithms is tested on the random graphs generated using NetworkX Erdős-Rényi model, a popular model for graph generation and Harwell–Boeing dataset, a well-known dataset for graph related problems. Further, we compare the results of both the meta-heuristic algorithms and the experimental results show that the proposed meta-heuristic algorithms for solving double Roman domination problem give a near optimal solution in reasonable time. Experimental results also show that the proposed ant colony optimization algorithm for solving double Roman domination problem outperforms genetic algorithm based procedure.

1. Introduction

Claude Berge in 1958 initiated the study of domination in graphs [1]. The concept of domination has been an extensively researched topic and one of the fastest growing branches of graph theory with many applications in real life, viz. computer communication networks, social network theory, wireless sensor networks and modelling biological networks [1,2].

Graphs $G(V, E)$ considered are undirected, connected and simple. Let $V(G)$ represent the vertex set and $E(G)$ represent the edge set of G. $N(v)$ represents the *open neighbourhood* of vertex v, $N(v) = \{u|(u, v) \in E\}$ and $N(v) \cup \{v\}$ represents the *closed neighbourhood* $N[v]$ of vertex v. We refer to [3] for undefined terminology and notations. For a graph G, a *dominating set* is defined as subset of vertices D such that each vertex of graph which is not in D is adjacent to at least one vertex in D. Domination problem of determining size of a smallest dominating set is known to be NP-hard [1] which implies no polynomial time algorithm exist for general graphs provided the conjecture P $\neq$ NP holds.

Roman domination was introduced in 2004, based on the defense strategies used to protect the Roman Empire [4]. Since then different variants of Roman domination parameters have been introduced to protect the region under different constraints and are surveyed in [5,6]. A function $h : V \to \{0, 1, 2\}$ on G with the condition that each vertex $u \in V$ with $h(u) = 0$ has a neighbour v with $h(v) = 2$ is known as the Roman dominating function of G. Optimization version of Roman domination problem is NP-hard [7].

In 2016, Beeler et al. introduced *double Roman domination* in [8]. A function $h : V \to \{0, 1, 2, 3\}$ of graph G such that each vertex $x \in V$ with $h(x) = 0$ is adjacent to at least two vertices y_1, y_2 with $h(y_1) = h(y_2) = 2$ or a vertex z_1 with $h(z_1) = 3$; also, a vertex p with $h(p) = 1$ is adjacent to at least one vertex q_1 with $h(q_1) \geq 2$ is known as *double Roman dominating function* (DROMDF) on graph G. $h(V) = \sum_{u \in V} h(u)$ is the weight of a DROMDF. The *double Roman domination number* $\gamma_{dR}(G)$ is the minimum weight of a DROMDF on G [8].

Consider a real-world problem of *deploying health care centers in a disease affected area* (set of regions). This problem can be modelled as a graph where each node represents a region and two nodes are adjacent if the nodes are very close so that faster transfer happen between two regions regarding equipment, medicine stock, etc. Due to limitation of resources, deployment can be done in such a way that each node has a health care center or it is adjacent to a node with two health

* Corresponding author.
E-mail addresses: aggarwal12himanshu@gmail.com (H. Aggarwal), pvsr@nitw.ac.in (P.V.S. Reddy).

https://doi.org/10.1016/j.asoc.2024.111306
Received 26 June 2023; Received in revised form 18 December 2023; Accepted 21 January 2024
Available online 1 February 2024

care centers. So, in case of emergency, the region has direct access to a health center or the help can be obtained from the region nearby with access to two centers. This problem can be modelled as a double Roman domination problem and hence is an application of the concept of double Roman domination.

Rest of the paper is organized into seven consecutive sections, namely, Literature Review, Genetic Algorithm for Double Roman Domination, Implementation using Three Heuristics, Ant Colony Optimization (ACO) Algorithm for Double Roman Domination, Implementation of ACO Algorithm, Experiments and Results and Conclusion.

2. Literature review

Optimization version of double Roman domination problem is defined as below.

Minimum double Roman domination problem (MDR)

Input: A graph G.

Output: $\gamma_{dR}(G)$.

MDR is known to be NP-hard [9] which implies no polynomial time algorithm exist for general graphs provided the conjecture P $\neq$ NP holds. MDR is NP-hard even for subclasses of bipartite graphs [10], chordal bipartite graphs [9], undirected path graphs [9], circle graphs [9] and planar graphs [11]. Deciding if $\gamma_{dr}(G) = 3\beta(G)$ is co-NP-complete for bipartite graphs, where $\beta(G)$ is the matching number of G [12]. However, MDR is polynomial time solvable for trees [13], cographs [14], unicyclic graphs [11], proper interval graphs [15], block graphs [9], interval graphs [9], P_4-free graphs [14] and chain graphs [10]. Different upper bounds on $\gamma_{dR}(G)$ are studied in [8,12,14,16–21] and lower bounds in [22]. Nordhaus–Gaddum type bounds on $\gamma_{dR}(G)$ is provided in [23]. Exact values for $\gamma_{dR}(G)$ when G is a path or a cycle is obtained in [24]. Stability and criticality concepts related MDR problem is studied in [25,26]. MDR problem is also studied on the generalized Petersen graphs [21] and generalized Sierpinski graphs [27]. Very recently an exhaustive survey on double Roman domination and its variants is presented in [28].

Different heuristic and meta-heuristic algorithms for solving domination problem have been proposed [29–31]. A meta-heuristic algorithm has been proposed for solving Roman domination problem [32]. Different integer linear programming formulations and two approximation algorithms for MDR problem are given in [10,33]. However no meta-heuristic algorithm exist for solving MDR problem, which motivated us to focus on this direction to solve different Roman domination variants. In this paper, we give a genetic algorithm and an ant colony optimization algorithm based solutions to solve MDR problem.

3. Genetic algorithm for double roman domination

In this section, first we describe the working of genetic algorithm.

Genetic algorithm is based on the principle of natural reproduction system. In this, two parent solutions from initial population participate in crossover & mutation and reproduce one child solution. Only the fittest solution will go to next generation [34]. Algorithm 1 gives the steps involved in genetic algorithm.

Genetic algorithm starts with creating *initial population* which is a set of feasible solutions in the current generation. Best individuals (solution) are selected based on their fitness evaluated using *fitness function* of the underlying problem which in turn generates intermediate population. Next a problem specific genetic *crossover operator* is used to generate new solution from those solutions to get the best solution. After that *mutation operator* is applied to maintain diversity among population individuals. Eventually initial population is replaced with the new population. Next we list the parameters associated with a genetic algorithm.

3.1. Generating parameters for genetic algorithm

Following are the parameters used in the Genetic algorithm.

- **Initial population** (*init_pop*): Genetic algorithm starts with generating initial population which is a set of feasible solutions.
- **Number of solutions** (*num_sol*): This parameter indicates the size of initial population i.e., number of feasible solutions.
- **Best solution obtained in current population** (*curr_best_sol*): It denotes the best solution among all the feasible solutions in the current generation.
- **Intermediate population**: After applying crossover operator and mutation a new intermediate population is obtained. Intermediate population will be the initial population for the next generation.
- **Best solution** (*best_sol*): Best solution is the near optimal solution obtained after algorithm terminates.
- **Termination condition**: The algorithm terminates either when there is no update on best_sol for consecutive number of generations or after the specified number of iterations.

Algorithm 1 Genetic Algorithm for MDR

Input: A simple, undirected graph G
Output: Near optimal solution
Initialize *num_sol*
Create *init_pop*
curr_best_sol ← solution with minimum cost on *init_pop*
best_sol ← *curr_best_sol*
while termination condition **do**
  init_pop ← Crossover and Mutation on *init_pop*
  curr_best_sol ← solution with minimum cost on *init_pop*
  if *cost*(*best_sol*) > *cost*(*curr_best_sol*) **then**
    best_sol ← *curr_best_sol*
  end if
end while
return best_sol

4. Implementation using three heuristics

We propose three heuristic methods to generate initial population for the genetic algorithm to solve MDR problem. The three heuristics are explained in detail in the following sections.

4.1. Heuristic 1

In this heuristic, we select a vertex v randomly from the graph, assign it label 3 and all its neighbours are assigned label 0. Next, the vertex v and its neighbours if any are removed from the graph. After removing the selected vertex v and its neighbours, if the remaining graph has a single vertex then it is assigned label 2. This three step process is repeated for the remaining graph until all vertices are labelled. Clearly, in this heuristic at most one vertex gets label 2.

Algorithm for Heuristic 1 is given in Algorithm 2 and its working is illustrated with the help of a graph in Fig. 1.

A graph $G(V, E)$ with vertex set $\{v_1, v_2, \ldots, v_n\}$ is provided as input to Algorithm 2. Next $S[v_1 \ldots v_n]$ is declared as a solution array and V' is initialized as V. A random vertex u from V' is picked and 3 is assigned to u and 0 to all its neighbours v i.e, $S(v)$ to 0. Next all those vertices which are labelled are removed from V'. If a single vertex u remains then 2 is assigned to u and u is removed from V'. It is easy to verify that S is a DROMDF of G with fitness value i.e., weight $\sum_{k=1}^{n} S(k)$.

For the graph shown in Fig. 1, $V' = \{v_1, v_2, v_3, v_4, v_5, v_6, v_7, v_8\}$. Suppose that Heuristic 1 picks vertex v_4 randomly from V'. Vertex v_4 is assigned label 3 i.e., $S(v_4) = 3$ and its neighbours are assigned label 0 i.e., $S(v_3) = 0$ and $S(v_5) = 0$. Now, we have $V' = \{v_1, v_2, v_6, v_7, v_8\}$ and $|V'| = |\{v_1, v_2, v_6, v_7, v_8\}| \neq 1$. If v_1 is the next vertex to be picked

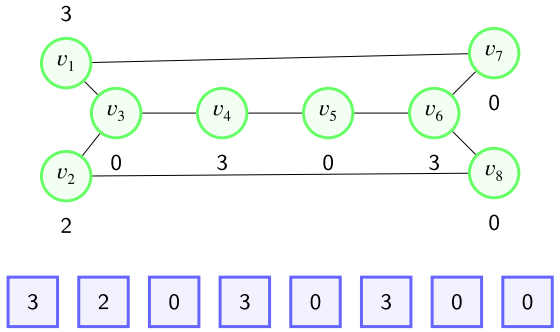

Fig. 1. A graph G labelled by Heuristic 1.

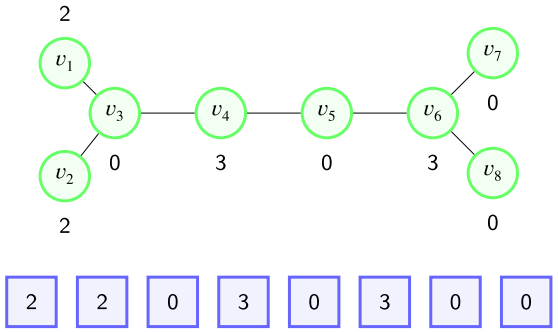

Fig. 2. A graph G labelled by Heuristic 2.

then set $S(v_1) = 3$ and assign 0 to its neighbour v_7 i.e., set $S(v_7) = 0$, so $V' = \{v_2, v_6, v_8\}$ and $|V'| \neq 1$. If v_6 is the next vertex to be picked then set $S(v_6) = 3$ and assign 0 to its neighbour v_8 i.e., set $S(v_8) = 0$, so $V' = \{v_2\}$ and $|V'| = 1$. Hence, we pick the remaining vertex v_2 and assign it label 2 i.e., $S(v_2) = 2$. Now $V' = \phi$. It is easy to verify that the labelling is a valid DROMDF of G with weight 11. We return S shown in Fig. 1 as a feasible solution.

4.2. Heuristic 2

This heuristic is similar to Heuristic 1 except for the fact that after removing the randomly selected vertex and its neighbours if the remaining graph has any isolated i.e., degree 0 vertices they are assigned label 2 and are also removed. The process is repeated for the remaining graph until all vertices are labelled. Clearly in this heuristic, more than one vertex might get label 2 unlike Heuristic 1 which assigns at most one vertex label 2.

Algorithm for Heuristic 2 is given in Algorithm 3 and its working is illustrated with the help of a graph in Fig. 2.

Algorithm 2 Heuristic 1

Input: A simple and undirected graph G
Output: Feasible solution S

```
Declare S
V' = V
while V' ≠ φ do
    Select a vertex u randomly from V' :
    S(u) = 3
    for all vertex vϵN(u) do
        S(v) = 0
    end for
    V' = V'\N[u]
    if |V'| = 1 then
        S(u) = 2 such that uϵV'
        V' = V'\{u}
    end if
end while
return S
```

For the graph shown in Fig. 2, $V' = \{v_1, v_2, v_3, v_4, v_5, v_6, v_7, v_8\}$. Suppose that Heuristic 2 picks vertex v_4 randomly from V'. Vertex v_4 is assigned label 3 i.e., $S(v_4) = 3$ and its neighbours are assigned label 0 i.e., $S(v_3) = 0$ and $S(v_5) = 0$. Now, we have $V' = \{v_1, v_2, v_6, v_7, v_8\}$. Since vertex v_1 and v_2 are the isolated vertex in the remaining graph, They are labelled as 2 and V' is updated as $V' = \{v_6, v_7, v_8\}$. Since there are no isolated vertices in the remaining graph, if v_6 is the next vertex to be picked then set $S(v_6) = 3$ and assign 0 to its neighbours v_7 and v_8 i.e., set $S(v_7) = 0$ and $S(v_8) = 0$. Now $V' = \phi$. It is easy to verify that the labelling is a valid DROMDF of G with weight 10. We return S shown in Fig. 2 as a feasible solution.

Algorithm 3 Heuristic 2

Input: A simple and undirected graph G
Output: Feasible solution S

```
Declare S
V' = V
while V' ≠ φ do
    Select a vertex u randomly from V' :
    S(u) = 3
    for all vertex vϵN(u) do
        S(v) = 0
    end for
    V' = V'\N[u]
    for all vertex uϵV' do
        if deg(u) = 0 then
            S(u) = 2
            V' = V'\{u}
        end if
    end for
end while
return S
```

4.3. Heuristic 3

In this heuristic, we take the advantage of Heuristics 2 and the degree of a vertex. Here, we select a vertex v with maximum degree from the graph by breaking ties arbitrarily. Selected vertex v is assigned label 3 and all its neighbours are assigned label 0. Next, the vertex v and its neighbours if any are removed from the graph. All isolated vertices of the remaining graph are assigned label 2 and these isolated vertices are removed from the remaining graph. The process is repeated for the new graph until all vertices are labelled. Algorithm for Heuristic 3 is given in Algorithm 4 and its working is illustrated with the help of a graph in Fig. 3. Algorithm 4 is similar to Algorithm 3 except for the fact that vertex with highest degree is selected after sorting the degrees of vertices of G in non-decreasing order.

For the graph shown in Fig. 3, $V' = \{v_1, v_2, v_3, v_4, v_5, v_6, v_7, v_8\}$. Clearly vertex v_6 has the maximum degree. Hence vertex v_6 is selected and assigned label 3 and its neighbours v_5, v_7 and v_8 are assigned label 0 i.e., $S(v_6) = 3$, $S(v_5) = 0$, $S(v_7) = 0$ and $S(v_8) = 0$. Now, we have $V' = \{v_1, v_2, v_3, v_4\}$. Since vertex v_1 is the only isolated vertex in the remaining graph, it is labelled as 2 and V' is updated as $V' = \{v_2, v_3, v_4\}$. Since v_3 is the highest degree vertex in the remaining graph it is assigned label 3 and its neighbours v_2 and v_4 are assigned label 0 i.e., $S(v_3) = 3$, $S(v_2) = 0$ and $S(v_4) = 0$. Now $V' = \phi$. It is easy to verify that the labelling is a valid DROMDF of G with weight 8. We return S shown in Fig. 3 as a feasible solution.

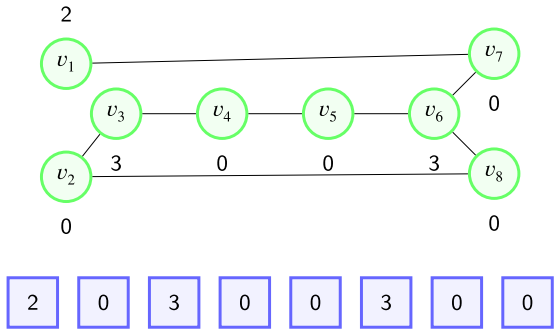

Fig. 3. A graph G labelled by Heuristic 3.

Algorithm 4 Heuristic 3

Input: A simple and undirected graph G
Output: Feasible solution S

```
Declare S
V' = V
Sort V' in descending order of their degree
while V' ≠ φ do
    Select first vertex u from V' such that :
    S(u) = 3
    for all vertex vϵN(u) do
        S(v) = 0
    end for
    V' = V'\N[u]
    for all vertex uϵV' do
        if deg(u) = 0 then
            S(u) = 2
            V' = V'\{u}
        end if
    end for
end while
return S
```

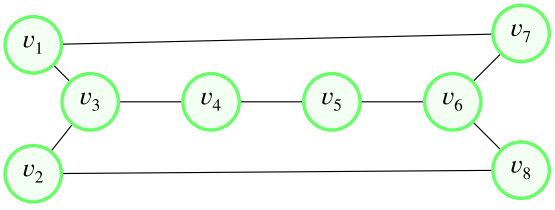

Fig. 4. Graph G.

4.4. Selection operator

Selection operator is applied to select best possible candidates for crossover and mutation. Consider a graph G shown in Fig. 4. Initial population i.e approximately 1000 feasible solutions is generated using three heuristics defined above for obtaining a feasible DROMDF of given graph. Next, we select two solutions among 1000 solution for performing crossover operator. Select the first solution S_1 using tournament selection operator i.e solution with minimum fitness value and second S_2 using roulette wheel i.e randomly select a solution out of remaining 999 solutions as shown in Fig. 5. After getting two solutions crossover and mutation operations are performed.

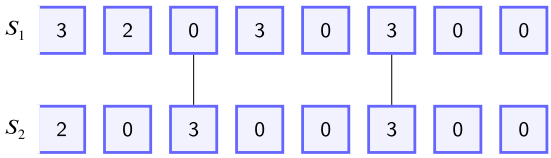

Fig. 5. Selected solutions S_1 and S_2.

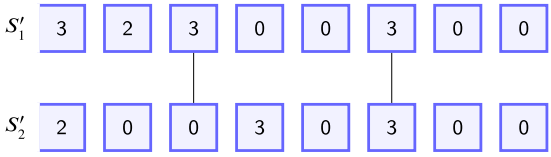

Fig. 6. Solutions S'_1 and S'_2 after crossover.

4.5. Crossover and mutation operator

Crossover and Mutation are the two important operations in genetic algorithm. Here, we use the two point crossover operation. First, we select two random vertices in first solution S_1 and swap the label values between those vertices of first solution S_1 with other solution S_2 which results in two child solutions S'_1, S'_2 generated after two point crossover are shown in Fig. 6 which may or may not be feasible. Next, we perform the feasibility test to make them feasible. S'_1 and S'_2 go through the feasibility check. Clearly S'_1 is feasible but not S'_2 as zero labelled vertex 2 is not adjacent to vertices with label three that is why zero labelled vertex 2 is assigned label two as shown in Fig. 7.

Algorithm 5 Crossover

Input: Two random solutions S_1 and S_2
Output: Child solution with better fitness value

```
Select two random vertices v1 and v2 from V
X ←set of vertex weights between v1 and v2 from S1
Y ←set of vertex weights between v1 and v2 from S2
Swap X and Y in S1 and S2
S'1 ← S1
S'2 ← S2
feasible(S'1)
feasible(S'2)
return (S'1, S'2)
```

5. Ant colony optimization algorithm for double Roman domination

In this section, first we describe the working of ant colony optimization (ACO) algorithm.

ACO algorithm is based on the principle of ants foraging behaviour. In this, ants explore their immediate surroundings, or their local search region, in search of food. Once they locate a region that is abundant in food, they share this information with other ants. In this manner, they utilize the local search neighbourhood to locate the best route. Algorithm 7 gives the steps involved in ACO algorithm.

ACO algorithm starts with creating feasible *initial solution* which is initially empty and initialize pheromone value. Every vertex of graph G is associated with pheromone value. At each iteration, each vertex make a probabilistic decision to update their values based on the pheromone value and local search mechanism. Repeat this process for fixed number of iterations. Eventually, the final solution will be feasible and might be close to the optimal solution. Next we list the parameters associated with a ACO algorithm.

Algorithm 6 Feasibility check

Input: Child solution C
Output: Modified feasible child solution
  for all vertex $u \epsilon C$ **do**
    if $S(u) = 0$ and $\nexists v \epsilon N(u)$ such that $S(v) = 3$ **then**
      $S(u) \leftarrow 2$
    end if
  end for

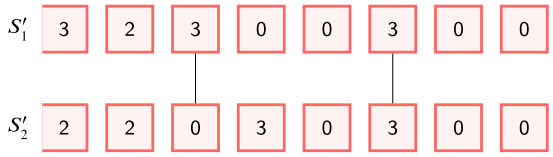

Fig. 7. Solutions S'_1 and S'_2 after attaining feasibility.

5.1. Generating parameters for ACO algorithm

- **Current solution** (S): It denotes the solution in current iteration which is initial empty (ϕ).
- **Current best solution** ($curr\_best\_sol$): It denotes the best solution in the current iteration and initialized with maximum integer possible (∞).
- **Best solution** ($best\_sol$): Best solution is a near optimal solution obtained as a result on termination of algorithm and initialized with maximum integer possible (∞).
- **Convergence Factor** ($conv\_fact$): It is a parametric constant which indicates the convergence of the algorithm which is initially 0.

6. Implementation of ACO algorithm

In this section, we propose ant colony optimization algorithm which consist of four major functions as follows:

- Construct solution($cons\_sol(S)$)
- Extend solution($ext\_sol(S)$)
- Reduce solution($red\_sol(S)$)
- Random variable neighbourhood search ($ran\_var\_neigh\_srch(S)$)

6.1. Pheromones update and convergence factor

The pheromone values ($\mathcal{T}$) of each vertex are updated in each iteration of total n iterations. It depends on the following parameters such as $curr\_best\_sol, best\_sol$ and their corresponding best fitness values $K_{curr\_best\_sol}, K_{best\_sol}$ respectively. $\mathcal{T}_u$ of vertex u is updated using equation 3 and this equation is implemented in update_pheromone($curr\_best\_sol$, $best\_sol$, $\mathcal{T}$). For constructing initial solution, the pheromone value of each vertex is initialized with 0.5 and the implementation of initialization is given in initialize_pheromone($\mathcal{T}$) as given in [35].

After updation, if for vertices with pheromone value less than 0.001 and greater than 0.999 then convergence factor ($conv\_fact$) is calculated using the below formula as:

$$conv\_fact = 2\left(\frac{\sum_{u\in V} max\{\mathcal{T}_{max} - \mathcal{T}_u, \mathcal{T}_u - \mathcal{T}_{min}\}}{n \cdot (\mathcal{T}_{max} + \mathcal{T}_{min})}\right) - 1 \tag{1}$$

Implementation details of Eq. (1) is given in $compute\_convergence(\mathcal{T})$.

$conv\_fact$ becomes zero if all pheromone values are 0.5. On the contrary, if all pheromone values are at either $\mathcal{T}_{min}$ or $\mathcal{T}_{max}$, $conv\_fact$ will be 1. In all other circumstances, its value lies between 0 and 1. This means that the algorithm will converge if the value of conv_fact increases. Hence, if $conv\_fact$ increases then pheromone values of each vertex are updated with 0.5.

Algorithm 7 ACO algorithm for MDR

Input: A simple, undirected graph G
Output: Near optimal solution
  Initialize $S \leftarrow \phi$, $conv\_fact \leftarrow 0$, $curr\_best\_sol \leftarrow \infty$, $best\_sol \leftarrow \infty$
  initialize_pheromone($\mathcal{T}$)
  Declare n as number of iterations
  while termination condition **do**
    for 1 … n **do**
      $S \leftarrow cons\_sol(S)$
      $S \leftarrow ext\_sol(S)$
      $S \leftarrow red\_sol(S)$
      $S \leftarrow ran\_var\_neigh\_srch(S)$
      $sum(S) \leftarrow \sum_{i=1}^{|V(G)|} S[i]$
      if $sum(S) < curr\_best\_sol$ **then**
        $curr\_best\_sol \leftarrow sum(S)$
      end if
    end for
    if $curr\_best\_sol < best\_sol$ **then**
      $best\_sol \leftarrow curr\_best\_sol$
    end if
      update_pheromone($curr\_best\_sol$, $best\_sol$, $\mathcal{T}$)
      $conv\_fact \leftarrow compute\_convergence(\mathcal{T})$
    if $conv\_fact > 0.99$ **then**
      initialize_pheromone($\mathcal{T}$)
    end if
  end while
  return $best\_sol$

6.2. Criteria for vertex selection

A random number (r) is chosen first between $0 < r < 1$. If $r \leq d_{\text{rate}}^{\text{ACO}}$ then select a vertex u from set of vertices V that maximizes the objective function $f(u) = deg(u) \cdot \mathcal{T}_u$, where $deg(u)$ is the degree of vertex u and $\mathcal{T}_u$ is the pheromone value associated with it, otherwise, a roulette wheel selection is made, with the probability of choosing a vertex u being

$$p_u = \frac{f(u)}{\sum_{u\in V} f(u)} \tag{2}$$

Implementation of Eq. (2) is given in choose_vertex(V).

6.3. Construct solution ($cons\_sol(S)$)

Construct solution is a function that is used to create the initial solution from the partial solution. Initially, solution is empty. Given a simple, undirected graph G, criteria for selecting a vertex u from the set of unlabelled vertices V' is given in Section 6.2. After vertex selection, set its label as 3 and all its neighbours as 0. Repeat this process until all the vertices are labelled.

Algorithm 8 Construct solution

Input: A simple, undirected graph G with solution S
Output: Feasible solution S
  $V' = V$
  while $V' \neq \phi$ **do**
    $u \leftarrow choose\_vertex(V')$
    $S(u) = 3$
    for all vertex $v \epsilon N(u)$ **do**
      $S(v) = 0$
    end for
    $V' = V' \backslash N[u]$
  end while
  return S

6.4. Extend solution ($ext\_sol(S)$)

Extend solution is a function that is used to extend the solution by choosing $|V_{02}|$ vertices where $|V_{02}|$ is the total number of vertices with label 0 or 2.

$$\mathcal{T}_u = \mathcal{T}_u + \rho \cdot \left(\frac{K_{curr\_best\_sol} \cdot \delta(curr\_best\_sol, u) + K_{best\_sol} \cdot \delta(best\_sol, u)}{K_{curr\_best\_sol} + K_{best\_sol}} - \mathcal{T}_u \right) \tag{3}$$

Select a vertex u from V_{02} and set its label as 3. $\lfloor r_{aug} \cdot |V_{02}| \rfloor$ determines the number of times this process is repeated. The criteria for selecting a vertex is given in section 6.2 but here d_{rate} is used in place of d_{rate}^{ACO} [35].

Algorithm 9 Extend solution

Input: A simple, undirected graph G with solution S
Output: Feasible solution S
  Declare V_{02}, itr
  Declare r_{aug} as parametric constant
  $V_{02} = V\ \{u : S(u) = 0 \text{ or } 2\}$
  $itr = r_{aug} * V_{02}$
  while $itr \neq 0$ **do**
    Select a vertex u from V_{02}
    $S[u] = 3$
    $V_{02} = V_{02} \backslash \{u\}$ $itr = itr - 1$
  end while
  return S

6.5. Reduce solution ($red\_sol(S)$)

Reduce solution is a function that is used to decrease the label of those vertices that have no impact on the feasibility of the solution i.e vertices with label 2 or 3 are decreased. First, sort the vertex set V in increasing order of their degree then select a vertex u from V and set its label 0, After modification if solution is not feasible then set its label to 2, again check for feasibility if not feasible then retain its original label. This process is repeated for all the vertices in V

Algorithm 10 Reduce Solution

Input: A simple, undirected graph G with Solution S
Output: Feasible solution S
  Declare init_lab
  $V' = V$
  Sort V' in increasing order of their degree
  while $V' \neq \phi$ **do**
    Select first vertex u from V' such that :
    if $S[u] = 3 \vee S[u] = 2$ **then**
      init_lab = $S[u]$
      $S[u] = 0$
      if !feasible(S) **then**
        $S[u] = 2$
        if !feasible(S) **then**
          $S[u]$ = init_lab
        end if
      end if
    end if
  end while
  return S

6.6. Destroy solution ($des\_sol(S)$)

Destroy solution is a function that is used to unlabelled some vertices with label 0 or 2. Number of vertices that are unlabelled depends on $\lceil n \cdot d \rceil$. Select a vertex u from V with label 0 or 2 and unlabel it (set its label as -1). Procedure for calculating the value of d is given in Eq. (4).

Algorithm 11 Destroy solution

Input: A simple, undirected graph G with Solution S
Output: Feasible solution S
  Declare V', itr
  $V' = V$
  $itr = |S| * d$
  while $itr \neq 0$ **do**
    Select a vertex u randomly from V'
    if $S[u] = 0 \vee S[u] = 2$ **then**
      $S[u] = -1$
    else
      $itr = itr + 1$
    end if
    $V' = V' \backslash \{u\}$
    $itr = itr - 1$
  end while
  return S

$$d = d_{min} + (k-1)\left(\frac{d_{max} - d_{min}}{k_{max} - 1} \right) \tag{4}$$

6.7. Random variable neighbourhood search

Random variable neighbourhood search is a function that is used to improve the best result obtained from ACO algorithm. It is an add-on feature to the ACO algorithm.

ran_var_neigh_srch searches in a region already discovered by ACO algorithm. It increases the chances of getting the solution which is close to the optimal solution. The *ran_var_neigh_srch* algorithm comprises five parameters: *k_max*, *d_min*, *d_max*, *max_noimpr*, and *max_itr*. The current solution S is first destroyed and then repaired in order to potentially improve upon it. Pseudocode for *ran_var_neigh_srch* algorithm is given in Algorithm 12. The repair steps of the *ran_var_neigh_srch* algorithm are same as *cons_sol* but instead of d_{rate}^{ACO}, d_{rate} is used as a parametric constant. If the solution obtained (S') is better than the current solution (S) then solution S is updated to S'. This process is repeated until the algorithm is terminated. The algorithm terminates, if *max_itr* iterations is reached or no improvement in solution for *max_noimpr* iterations i,e $c_{noimpr} < max_{noimpr}$.

7. Experiments and results

Implementation details and the results obtained are discussed in this section. Proposed genetic algorithm is implemented in Python Language and all the experiments are performed on 11th Gen Intel(R) Core(TM) i5-1135G7 @2.40 GHz –1.38 GHz system with 8 GB RAM. Performance of the proposed genetic algorithm is tested on random graphs generated using Erdös-Rényi model as it is used by most of the researchers working in this domain [36].

This model generates graph G based on number of vertices (n) and probability (p) of each edge in a Graph G and we have also tested the performance of both genetic and ACO algorithms in the Harwell–Boeing dataset available at https://math.nist.gov/MatrixMarket/matrices.html, a well-known dataset for graph related problems. In [37] double Roman domination number of paths and cycles has been obtained and is shown in Table 1.

We first tested the performance of the proposed genetic algorithm on path and cycle graphs for 15 to 100 vertices. The results obtained are given in Table 2. It is observed that as the number of iterations i.e., generations is increased the answer given by our algorithm is close

Algorithm 12 Random variable neighbourhood search

Input: A simple, undirected graph G with Solution S
Output: Feasible solution S

```
Initialize k, k_max, c_noimpr, max_noimpr, max_itr
Declare S'
S' = S
while c_noimpr < max_noimpr and max_itr ≠ 0 do
    des_sol(S')
    cons_sol(S')
    ext_sol(S')
    red_sol(S')
  if sum(S') < sum(S) then
        S = S'
        k = 1
        c_noimpr = 0
  else
        k++
        c_noimpr ++
  end if
  if k > k_max then
        k = 1
  end if
    max_itr = max_itr − 1
end while
return S
```

Table 1
Some Standard Graphs with known $\gamma_{dr}(G)$.

Graph	Optimal results
Path	n, if $n \equiv 0 \pmod 3$ $n+1$, if $n \equiv 1$ or $2 \pmod 3$
Cycle	n, if $n \equiv 0, 2, 3, 4 \pmod 6$) $n+1$, if $n \equiv 1, 5 \pmod 6$

to the optimal solution. Next, the performance of the proposed genetic algorithm is tested on random graphs up to 500 vertices generated using Erdös-Rényi model denoted as G(n, p) where n is number of vertices and p is the probability of each edge between vertices in a graph G and Harwell–Boeing collection of graphs with up to 3345 vertices for 100000 iterations and the results are shown in Tables 5 and 7. Results given under the Heuristic 1 column indicate the solution to the MDR problem obtained for different graphs using the proposed genetic algorithm when the initial population has been generated using only Heuristic 1, as proposed in Section 4. Similarly, Heuristic 2 and Heuristic 3 columns indicate the results obtained when the initial population has been generated using Heuristic 2 and Heuristic 3, respectively. From Tables 5 and 7, it is evident that the results obtained for Heuristics 1 and 2 are better than Heuristic 3. Hence we also have tested the performance of the proposed genetic algorithm by generating 40%, 40% and 20% of initial population using heuristic 1, heuristic 2 and heuristic 3 respectively and the obtained results are shown in the last column of Tables 5 and 7. Since optimal solution for these graphs is not known and no meta-heuristic algorithm has been proposed in the literature to solve MDR problem, we have considered the upper bounds [14] and lower bounds [22] as shown in Table 3 and found that obtained results are within the bounds. From Tables 5 and 7 it is clear that results obtained when the initial population is generated by combining three heuristics for the mentioned percentages is better. Comparison of GA method with ACO algorithm on 20 individual runs for different HB graphs is shown in Table 6, which emphasizes the observation that ACO based solution outperforms GA based solution for solving MDR problem.

Table 2
Results obtained for path and cycle graphs using GA.

Graph	Optimal results	$\gamma_{dR}(G)$	Obtained Results		
			# of iterations		
			10 000	100 000	1 000 000
Path	15	15	15	15	15
	32	33	34	33	33
	68	69	75	72	70
	100	101	115	112	108
Cycle	15	15	15	15	15
	32	32	33	32	32
	67	68	76	73	71
	100	100	114	112	110

Table 3
Lower and Upper Bounds on $\gamma_{dr}(G)$.

Graph parameters	Lower bound	Upper bound
$\lvert V(G)\rvert, \delta, \Delta$	$\lceil \frac{3\lvert V(G)\rvert}{\Delta+1} \rceil$	$\frac{3\lvert V(G)\rvert(1+\ln\frac{2(1+\delta)}{3})}{1+\delta}$

Next, we tested and compared the performance of ACO algorithm with the genetic algorithm on Harwell–Boeing dataset and the results are shown in Table 8. For implementation of ACO algorithm the parametric constants used are listed in Table 4 along with the description of the parameters [35].

Based on the comparison results given in Table 8, it is observed that even for one outer iteration and five inner iterations the proposed ant colony optimization algorithm for MDR problem outperforms GA. The results provided in Tables 5 and 7 are for single run of the algorithm. We employ the sign test to assess the presence of a significant difference in the performance between Genetic Algorithm (GA) and the Ant Colony Optimization (ACO) algorithm (see Table 6).

- *Null Hypothesis*: There is no difference between the performance of GA and ACO in solving MDR problem.
- *Alternative Hypothesis*: ACO algorithm consistently outperforms GA algorithm in solving MDR problem.
- *Counting Signs*: Count the number of positive and negative signs based on the observed differences as shown in Table 8 which includes 95 wins (+), 1 negatives (-) and 16 ties (0).
- *Critical Value Determination*: Utilizing a binomial distribution, we find the critical value associated with our data. We conducted tests on a total of 112 graphs, with 16 of them resulting in ties. Therefore, the effective number of trials considered in the analysis is 96 with minimum of 95 and 1 i.e., 1 success and 0.5 for the probability of accepting the null hypothesis. After applying binomial distribution

$$\binom{96}{0}(0.5)^{96}(0.5)^{0} + \binom{96}{1}(0.5)^{95}(0.5)^{1},$$

we get the critical value as $1.224312124\times10^{-27}$.
- *Hypothesis Decision*: Comparing the critical value to a predetermined alpha level of 0.05, we find that the critical value $1.224312124\times10^{-27}$ which is very smaller. Consequently, we reject the null hypothesis, suggesting a significant difference in the performance of GA and ACO and accept the alternative hypothesis that "ACO algorithm consistently outperforms GA algorithm in solving MDR problem".

8. Conclusion

In this paper, we have proposed a genetic algorithm and an ant colony optimization algorithm based solution for solving the double

Table 4
Parametric constants used in ACO algorithm.

Parameter	Description	Value
n	Number of solution constructions per iteration in ACO	5
ρ	Evaporation rate of ACO	0.2
d_{rate}	A constant for choosing vertices in the Extend Solution (also used in Random Variable Neighbourhood Search)	0.9
d_{rate}^{ACO}	A constant for selecting vertices in the Construct Solution	0.7
d_{min}	Minimum value of destruction rate applied within the neighbourhood function context in Random Variable Neighbourhood Search	0.2
d_{max}	Maximum value of destruction rate applied within the neighbourhood function context in Random Variable Neighbourhood Search	0.5
r_{aug}	Constant parameter to iteratively add the number of vertices in each step of the Extend Solution (also used in Random Variable Neighbourhood Search)	0.05
k_{max}	Number of neighbourhood functions in Random Variable Neighbourhood Search	5
max_{itr}	Maximum number of iteration for Random Variable Neighbourhood Search	150
max_{noimpr}	Maximum number of consecutive iterations without improvement of current solution used in Random Variable Neighbourhood Search	10

Table 5
Solution to MDR problem obtained for different graphs generated using Erdös-Rényi model.

n	p	LB	UB	Heuristic 1	Heuristic 2	Heuristic 3	Combined
25	0.2	9	48	18	18	21	18
	0.5	5	23	8	8	15	8
	0.8	4	15	6	6	6	6
50	0.2	9	60	23	24	30	23
	0.5	5	30	9	9	18	9
	0.8	4	18	6	6	12	6
75	0.2	10	75	26	30	27	26
	0.5	5	30	11	12	15	12
	0.8	4	19	6	6	6	6
100	0.2	9	81	30	30	36	30
	0.5	5	34	14	12	18	12
	0.8	5	17	6	6	12	6
200	0.2	11	86	39	39	36	35
	0.4	6	35	14	15	24	15
	0.5	4	23	6	9	9	8
300	0.2	11	98	44	45	57	45
	0.4	6	38	17	18	18	15
	0.5	4	24	9	9	15	9
400	0.2	11	93	47	45	54	48
	0.4	6	41	17	18	24	20
	0.5	4	25	9	9	12	9
500	0.2	12	95	50	48	60	50
	0.4	6	40	18	18	24	18
	0.5	4	26	9	9	12	9

Table 6
Comparison of genetic algorithm with ACO algorithm on 20 individual runs for different HB graphs.

Graphs	Genetic Algorithm	ACO	Graphs	Genetic Algorithm	ACO
662_bus	687	580	bcsstk01	23	18
ash219	248	130	bcsstk02	3	3
ash292	165	132	bcsstk03	72	66
ash331	408	188	bcsstk04	30	24
ash608	495	328	can_144	36	36
ash85	56	47	can_161	78	57
bcspwr01	43	36	can_187	100	66
bcspwr02	54	43	can__24	12	12
bcspwr03	123	97	can__61	24	18
bcspwr04	270	195	can__73	69	45
bcspwr05	480	398	ck104	38	36

Roman domination problem which is NP-hard. Since no meta-heuristic algorithm has been proposed in the literature for this problem, effectiveness of the proposed genetic algorithm is tested on the random graphs generated using NetworkX Erdős-Rényi model, a popular model for graph generation and found that the results obtained are close to the known lower bounds for the problem. Both the algorithms are also tested on the Harwell–Boeing collection of graphs, a popular dataset for graph related problems and found that the results for both the algorithms are within the known bounds for the problem. This fact emphasizes that the algorithms are efficient. Additionally, the sign test performed proves that the ant colony optimization algorithm performs significantly better than the genetic algorithm for solving the double Roman domination problem. Designing better meta-heuristic algorithms for the double Roman domination problem remains open.

CRediT authorship contribution statement

Himanshu Aggarwal: Conceptualization, Investigation, Methodology, Software, Validation, Writing – original draft. **P. Venkata Subba Reddy:** Conceptualization, Methodology, Resources, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

https://math.nist.gov/MatrixMarket/matrices.html.

Acknowledgement

We thank the reviewers for their careful and detailed reading of our manuscript and for their insightful comments and suggestions, which led to significant improvements in the paper.

Table 7
Results obtained for HB graphs.

Graphs	n	LB	UB	Heuristic 1	Heuristic 2	Heuristic 3	Combined
662_bus	662	199	1278	833	849	687	687
add20	2395	58	4625	2879	2877	2757	2757
ash219	219	55	370	239	243	264	258
ash292	292	63	433	171	171	165	164
ash331	331	67	560	428	396	417	401
ash608	608	122	1029	875	849	495	487
ash85	85	26	143	56	57	69	57
ash958	958	180	1622	1409	1428	690	690
bcspwr01	39	20	75	44	42	45	42
bcspwr02	49	21	94	53	54	60	53
bcspwr03	118	36	227	125	128	123	122
bcspwr04	274	52	529	284	288	270	270
bcspwr05	443	133	855	566	564	480	480
bcspwr06	1454	336	2808	1988	1980	1650	1650
bcspwr07	1612	372	3113	2246	2205	1848	1848
bcsstk01	48	12	63	23	24	27	18
bcsstk02	66	3	14	3	3	3	3
bcsstk03	112	56	166	71	72	84	72
bcsstk04	132	9	87	30	30	42	30
blckhole	2132	320	3802	1463	1454	1552	1456
bp___0	822	10	1391	767	780	762	737
can_1054	1054	91	1257	563	573	333	333
can_1072	1072	92	1279	578	594	348	348
can_144	144	29	171	62	72	36	36
can_161	161	29	192	72	81	99	81
can_187	187	57	247	101	99	135	101
can__24	24	8	35	12	12	18	12
can__61	61	8	80	24	24	24	23
can__73	73	25	123	68	69	75	62
cavity01	317	16	294	56	72	27	27
cavity02	317	16	294	71	63	27	27
cavity03	317	16	294	71	75	27	27
cavity04	317	16	294	62	66	27	27
cdde1	961	577	1627	995	986	1443	998
cdde2	961	577	1627	994	999	1443	980
cdde3	961	577	1627	995	999	1443	996
cdde4	961	577	1627	986	979	1443	988
cdde5	961	577	1627	998	1003	1443	1002
cdde6	961	577	1627	1001	996	1443	1001
ch400	400	120	772	264	264	291	266
ck104	104	32	124	36	36	42	36
ck656	656	197	1267	527	528	579	531
curtis54	54	11	91	35	33	36	32
dwt_162	162	54	312	92	96	111	95
dwt_193	193	20	193	47	48	57	50
dwt_198	198	50	382	120	126	153	126
dwt_209	209	37	310	116	123	123	116
dwt_221	221	56	328	131	132	123	123
dwt_245	245	57	473	218	219	234	218
dwt_310	310	85	460	174	174	195	174
dwt_346	346	55	668	242	240	252	237
dwt_361	361	121	536	197	199	243	200
dwt_419	419	97	499	242	240	246	242
dwt_492	492	135	833	359	363	339	332
dwt_503	503	61	747	236	237	246	236
dwt_512	512	103	913	422	425	425	411
dwt_918	918	212	1363	572	543	510	509
dwt_992	992	166	994	267	270	360	261
dwt__59	59	30	113	50	51	60	53
dwt__66	66	33	127	45	48	57	48
dwt__72	72	44	139	81	83	96	80
dwt__87	87	21	168	65	63	63	59
e05r0000	236	12	219	50	48	27	27
e05r0100	236	12	219	47	42	27	27
e05r0200	236	12	219	47	45	27	27
e05r0300	236	12	219	50	51	27	27
e05r0400	236	12	219	44	48	27	27
eris1176	1176	36	2271	1169	1161	1020	1020
fidap001	216	13	216	68	66	105	71
fidap002	441	11	185	33	33	36	33
fidap004	1601	130	1910	641	657	264	264
fidap005	27	6	32	9	9	9	9
fs1831	183	6	353	119	144	159	142
fs_183_1	183	6	353	116	120	159	143

(*continued on next page*)

Table 7 (*continued*).

Graphs	*n*	LB	UB	Heuristic 1	Heuristic 2	Heuristic 3	Combined
fs_183_3	183	6	353	143	117	159	143
fs_183_4	183	6	353	140	114	159	116
fs_183_6	183	6	353	158	144	159	144
fs_541_1	541	3	645	3	3	3	3
fs_541_2	541	3	645	3	3	3	3
fs_680_1	680	146	1313	1037	1053	837	836
fs_760_1	760	95	1129	533	519	480	460
gent113	113	13	201	95	87	117	90
gre_1107	1107	277	1644	767	759	846	765
gr_30_30	900	300	1337	488	492	675	497
hor_131	434	32	644	221	219	234	224
ibm32	32	8	47	20	21	24	21
illc1033	1033	11	1534	1205	1221	519	422
impcol_b	59	10	99	29	33	57	33
impcol_c	137	28	231	113	111	90	85
impcol_d	425	80	719	365	372	384	357
jagmesh1	936	402	1390	644	639	684	641
jagmesh2	1009	433	1498	674	684	756	674
jgl009	9	3	10	3	3	3	3
jgl011	11	3	14	3	3	3	3
lnsp_131	131	31	233	155	148	169	154
lns_131	131	31	233	160	154	169	151
lop163	163	55	215	107	108	255	113
lshp1009	1009	433	1498	677	684	756	683
lshp_265	265	114	393	173	171	201	176
lshp_406	406	174	603	269	270	270	267
lund_a	147	21	194	38	39	57	39
lund_b	147	21	194	39	39	57	39
nos1	237	143	457	281	279	351	282
nos2	957	575	1848	1190	1186	1431	1191
nos4	100	43	193	86	84	117	86
pores_1	30	9	35	12	12	24	12
rgg010	10	3	8	3	3	3	3
saylr1	238	143	402	233	231	357	227
saylr3	1000	429	1783	1646	1326	1476	133
shl_200	663	5	1122	683	471	450	436
shl_400	663	5	1122	665	495	477	469
shl___0	663	5	1122	488	486	462	445
sstmodel	3345	558	5966	3788	3169	3177	3158
steam1	240	35	223	69	69	126	72
str200	363	22	701	275	270	294	272
will199	199	43	336	143	144	177	146
will57	57	16	110	38	36	33	32
young1c	841	505	1423	875	855	1263	876

Table 8
Comparison of genetic algorithm with ACO algorithm.

Graphs	GA	ACO			Sign
		# of iterations			
		1	10	20	
662_bus	687	608	592	580	+ve
ash219	258	132	131	130	+ve
ash292	164	134	136	132	+ve
ash331	401	190	188	188	+ve
ash608	487	334	332	328	+ve
ash85	57	47	47	47	+ve
ash958	690	517	516	515	+ve
bcspwr01	42	39	37	36	+ve
bcspwr02	53	43	43	43	+ve
bcspwr03	122	101	96	97	+ve
bcspwr04	270	197	194	195	+ve
bcspwr05	480	412	405	398	+ve
bcsstk01	18	20	18	18	0
bcsstk02	3	3	3	3	0
bcsstk03	72	66	65	66	+ve
bcsstk04	30	24	24	24	+ve
bp___0	737	405	399	399	+ve
can_1054	333	234	234	234	+ve
can_1072	348	237	234	234	+ve
can_144	36	36	36	36	0
can_161	81	57	57	57	+ve
can_187	101	66	69	66	+ve
can__24	12	12	12	12	0

(*continued on next page*)

Table 8 (*continued*).

Graphs	GA	ACO			Sign
		# of iterations			
		1	10	20	
can__61	23	18	18	18	+ve
can__73	62	46	45	45	+ve
cavity01	27	27	27	27	0
cavity02	27	27	27	27	0
cavity03	27	27	27	27	0
cavity04	27	27	27	27	0
cdde1	998	866	828	834	+ve
cdde2	980	861	828	829	+ve
cdde3	996	856	834	837	+ve
cdde4	988	860	845	833	+ve
cdde5	1002	862	842	841	+ve
cdde6	1001	872	840	834	+ve
ch400	266	244	233	230	+ve
ck104	36	36	36	36	0
ck656	531	480	465	458	+ve
curtis54	32	31	31	31	+ve
dwt_162	95	84	80	82	+ve
dwt_193	50	38	36	36	+ve
dwt_198	126	84	84	84	+ve
dwt_209	116	92	90	87	+ve
dwt_221	123	101	96	92	+ve
dwt_245	218	170	168	169	+ve
dwt_310	174	140	131	131	+ve
dwt_346	237	183	181	177	+ve
dwt_361	200	156	155	154	+ve
dwt_419	242	180	177	174	+ve
dwt_492	332	269	264	260	+ve
dwt_503	236	170	164	163	+ve
dwt_512	411	347	342	337	+ve
dwt_918	509	350	341	340	+ve
dwt_992	261	210	204	–	+ve
dwt__59	53	46	44	44	+ve
dwt__66	48	42	42	41	+ve
dwt__72	80	76	73	72	+ve
dwt__87	59	49	48	49	+ve
e05r0000	27	27	27	27	+ve
e05r0100	27	27	27	27	+ve
e05r0200	27	27	27	27	+ve
e05r0300	27	27	27	27	+ve
e05r0400	27	27	27	27	+ve
eris1176	1020	781	777	–	+ve
fidap001	71	36	35	35	+ve
fidap002	33	24	18	–	+ve
fidap004	264	285	287	–	-ve
fidap005	9	9	9	9	0
fs1831	142	52	52	52	+ve
fs_183_1	143	52	52	52	+ve
fs_183_3	143	52	52	52	+ve
fs_183_4	116	52	52	52	+ve
fs_183_6	144	52	52	52	+ve
fs_541_1	3	3	3	3	0
fs_541_2	3	3	3	3	0
fs_680_1	836	458	455	453	+ve
fs_760_1	460	286	279	276	+ve
gent113	90	60	60	59	+ve
gre_1107	765	704	691	675	+ve
gr_30_30	497	398	369	370	+ve
hor__131	224	185	162	161	+ve
ibm32	21	21	19	19	+ve
illc1033	422	39	39	–	+ve
impcol_b	33	29	26	26	+ve
impcol_c	85	78	77	77	+ve
impcol_d	357	210	210	204	+ve
jagmesh1	641	545	534	526	+ve
jagmesh2	674	606	576	582	+ve
jgl009	3	3	3	3	0
jgl011	3	3	3	3	0
lnsp_131	154	113	112	111	+ve
lns__131	151	113	112	112	+ve
lop163	113	71	71	68	+ve
lshp1009	683	598	586	568	+ve
lshp_265	176	151	146	140	+ve

(*continued on next page*)

Table 8 (*continued*).

Graphs	GA	ACO			Sign
		# of iterations			
		1	10	20	
lshp_406	267	228	226	217	+ve
lund_a	39	38	36	35	+ve
lund_b	39	38	36	35	+ve
nos1	282	206	205	205	+ve
nos2	1191	859	853	–	+ve
nos4	86	71	68	68	+ve
pores_1	12	15	12	12	0
rgg010	3	3	3	3	0
saylr1	227	202	197	189	+ve
shl_200	436	305	306	308	+ve
shl_400	469	331	319	319	+ve
shl__0	445	317	315	310	+ve
steam1	72	60	51	54	+ve
str200	272	168	159	162	+ve
will199	146	106	101	104	+ve
will57	32	27	26	26	+ve
young1c	876	771	724	728	+ve

References

[1] Teresa W. Haynes, Stephen Hedetniemi, Peter Slater, Fundamentals of Domination in Graphs, CRC Press, 1998.

[2] Teresa W. Haynes, Domination in Graphs: Volume 2: Advanced Topics, Routledge, 2017.

[3] Douglas Brent West, Introduction to Graph Theory, vol. 2, Prentice Hall, Upper Saddle River, NJ, 2001.

[4] Ernie J. Cockayne, Paul A. Dreyer Jr., Sandra M. Hedetniemi, Stephen T. Hedetniemi, Roman domination in graphs, Discrete Math. 278 (1–3) (2004) 11–22.

[5] Mustapha Chellali, N. Jafari Rad, Seyed Mahmoud Sheikholeslami, Lutz Volkmann, Varieties of roman domination II, AKCE Int. J. Graphs Comb. 17 (3) (2020) 966–984.

[6] M. Chellali, N. Jafari Rad, S.M. Sheikholeslami, L. Volkmann, Varieties of roman domination, in: Structures of domination in graphs, Springer, 2021, pp. 273–307.

[7] M. Liedloff, T. Kloks, J. Liu, S.H. Peng, Roman domination in some special classes of graphs, Technical report Report TR-MA-04-01, 2004.

[8] Robert A. Beeler, Teresa W. Haynes, Stephen T. Hedetniemi, Double Roman domination, Discrete Appl. Math. 211 (2016) 23–29.

[9] S. Banerjee, Michael A. Henning, Dinabandhu Pradhan, Algorithmic results on double Roman domination in graphs, J. Comb. Optim. 39 (1) (2020) 90–114.

[10] Chakradhar Padamutham, Venkata Subba Reddy Palagiri, Complexity of Roman {2}-domination and the double Roman domination in graphs, AKCE Int. J. Graphs Comb. 17 (3) (2020) 1081–1086.

[11] Abolfazl Poureidi, Nader Jafari Rad, On algorithmic complexity of double Roman domination, Discrete Appl. Math. 285 (2020) 539–551.

[12] Lyes Ouldrabah, Lutz Volkmann, An upper bound on the double Roman domination number, Bull. Iran. Math. Soc. 47 (2021) 1315–1323.

[13] Xiujun Zhang, Zepeng Li, Huiqin Jiang, Zehui Shao, Double Roman domination in trees, Inf. Process. Lett. 134 (2018) 31–34.

[14] Jun Yue, Meiqin Wei, Min Li, Guodong Liu, On the double Roman domination of graphs, Appl. Math. Comput. 338 (2018) 669–675.

[15] Abolfazl Poureidi, A linear algorithm for double Roman domination of proper interval graphs, Discrete Math. Algorithms Appl. 12 (01) (2020) 2050011.

[16] Jafar Amjadi, Sakineh Nazari-Moghaddam, Seyed Mahmoud Sheikholeslami, Lutz Volkmann, An upper bound on the double Roman domination number, J. Comb. Optim. 36 (2018) 81–89.

[17] Xue-gang Chen, A note on the double Roman domination number of graphs, Czechoslovak Math. J. 70 (1) (2020) 205–212.

[18] Michael A. Henning, Nader Jafari Rad, A characterization of double Roman trees, Discrete Appl. Math. 259 (2019) 100–111.

[19] Rana Khoeilar, Hossein Karami, Mustapha Chellali, Seyed Mahmoud Sheikholeslami, An improved upper bound on the double Roman domination number of graphs with minimum degree at least two, Discrete Appl. Math. 270 (2019) 159–167.

[20] Saeed Kosari, Zehui Shao, Seyed Mahmoud Sheikholeslami, Mustapha Chellali, Rana Khoeilar, Hossein Karami, Double Roman domination in graphs with minimum degree at least two and no c 5-cycle, Graphs Combin. 38 (2) (2022) 39.

[21] Zehui Shao, Pu Wu, Huiqin Jiang, Zepeng Li, Janez Žerovnik, Xiujun Zhang, Discharging approach for double Roman domination in graphs, IEEE Access 6 (2018) 63345–63351.

[22] Nader Jafari Rad, Hadi Rahbani, Some progress on the double Roman domination in graphs, Discuss. Math. Graph Theory 39 (1) (2019) 41–53.

[23] Lutz Volkmann, Double Roman and double Italian domination, Discuss. Math. Graph Theory (2021).

[24] Hossein Abdollahzadeh Ahangar, Mustapha Chellali, Seyed Mahmoud Sheikholeslami, On the double Roman domination in graphs, Discrete Appl. Math. 232 (2017) 1–7.

[25] Sakineh Nazari-Moghaddam, Lutz Volkmann, Critical concept for double Roman domination in graphs, Discrete Math. Algorithms Appl. 12 (02) (2020) 2050020.

[26] Pu Wu, Zehui Shao, Huiqin Jiang, S. Nazari-Moghaddam, Seyed Mahmoud Sheikholeslami, Double Roman stable graphs, Ars Combin. 152 (2020) 183–200.

[27] V. Anu, S. Aparna Lakshmanan, The double Roman domination number of generalized Sierpiński graphs, Discrete Math. Algorithms Appl. 12 (04) (2020) 2050047.

[28] Darja Rupnik Poklukar, Janez Žerovnik, Double Roman domination: A survey, Mathematics 11 (2) (2023) 351.

[29] Sachchida Nand Chaurasia, Alok Singh, A hybrid evolutionary algorithm with guided mutation for minimum weight dominating set, Appl. Intell. 43 (3) (2015) 512–529.

[30] Cu Nguyen Giap, Dinh Thi Ha, Parallel genetic algorithm for minimum dominating set problem, in: 2014 International Conference on Computing, Management and Telecommunications, ComManTel, IEEE, 2014, pp. 165–169.

[31] Yiyuan Wang, Shaowei Cai, Jiejiang Chen, Minghao Yin, A fast local search algorithm for minimum weight dominating set problem on massive graphs, in: IJCAI, 2018, pp. 1514–1522.

[32] Aditi Khandelwal, Kamal Srivastava, Gur Saran, On roman domination of graphs using a genetic algorithm, in: Computational Methods and Data Engineering: Proceedings of ICMDE 2020, Volume 1, Springer, 2020, pp. 133–147.

[33] Qingqiong Cai, Neng Fan, Yongtang Shi, Shunyu Yao, Integer linear programming formulations for double Roman domination problem, Optim. Methods Softw. 37 (1) (2022) 1–22.

[34] John H. Holland, Adaptation in Natural and Artificial Systems: An Introductory Analysis with Applications to Biology, Control, and Artificial Intelligence, MIT Press, 1992.

[35] Salim Bouamama, Christian Blum, Jean-Guillaume Fages, An algorithm based on ant colony optimization for the minimum connected dominating set problem, Appl. Soft Comput. 80 (2019) 672–686.

[36] Laura A. Sanchis, Experimental analysis of heuristic algorithms for the dominating set problem, Algorithmica 33 (2002) 3–18.

[37] V. Anu, Aparna Lakshmanan, Double roman domination number, Discrete Appl. Math. 244 (2018) 198–204.